A proof of concept tool to verify estimates

Terence Tao · 1 May 2025

https://terrytao.wordpress.com/2025/05/01/a-proof-of-concept-tool-to-verify-estimates/

tao2typ · converted to Typst 2026-10-14 (UTC)

This post was inspired by some recent discussions with Bjoern Bringmann.

Symbolic math software packages are highly developed for many mathematical tasks in areas such as algebra, calculus, and numerical analysis. However, to my knowledge we do not have similarly sophisticated tools for verifying asymptotic estimates – inequalities that are supposed to hold for arbitrarily large parameters, with constant losses. Particularly important are functional estimates, where the parameters involve an unknown function or sequence (living in some suitable function space, such as an L^p space); but for this discussion I will focus on the simpler situation of asymptotic estimates involving a finite number of positive real numbers, combined using arithmetic operations such as addition, multiplication, division, exponentiation, and minimum and maximum (but no subtraction). A typical inequality here might be the weak arithmetic mean-geometric mean inequality

$$(abc)^{1/3} \lesssim a+b+c \qquad (1)$$

where a, b, c are arbitrary positive real numbers, and the $\lesssim$ here indicates that we are willing to lose an unspecified (multiplicative) constant in the estimates.

I have wished in the past (e.g., in this MathOverflow answer) for a tool that could automatically determine whether such an estimate was true or not (and provide a proof if true, or an asymptotic counterexample if false). In principle, simple inequalities of this form could be automatically resolved by brute force case splitting. For instance, with (1), one first observes that $a+b+c$ is comparable to $\max(a,b,c)$ up to constants, so it suffices to determine if

$$(abc)^{1/3} \lesssim \max(a,b,c). \qquad (2)$$

Next, to resolve the maximum, one can divide into three cases: $a \gtrsim b, c$; $b \gtrsim a, c$; and $c \gtrsim a, b$. Suppose for instance that $a \gtrsim b, c$. Then the estimate to prove simplifies to

$$(abc)^{1/3} \lesssim a,$$

and this is (after taking logarithms) a positive linear combination of the hypotheses $a \gtrsim b$, $a \gtrsim c$. The task of determining such a linear combination is a standard linear programming task, for which many computer software packages exist.

Any single such inequality is not too difficult to resolve by hand, but there are applications in which one needs to check a large number of such inequalities, or split into a large number of cases. I will take an example at random from an old paper of mine (adapted from the equation after (51), and ignoring some epsilon terms for simplicity): I wanted to establish the estimate

$$\frac{\langle N_2\rangle^{1/2}}{\langle N_1\rangle^{1/4} L_1^{1/2} L_2^{1/2}} L_{\min}^{1/2} N^{-1} (N_1 N_2 N_3)^{1/2} \lesssim 1 \qquad (3)$$

for any $N_1, N_2, N_3, L_1, L_2, L_3 > 0$ obeying the constraints

$$N_{\max} \sim N_{\mathrm{med}} \sim N; \quad L_{\max} \sim L_{\mathrm{med}} \gtrsim N_1 N_2 N_3$$

where $N_{\max}$, N_{med}, and $N_{\min}$ are the maximum, median, and minimum of N_1, N_2, N_3 respectively, and similarly for $L_{\max}$, L_{med}, and $L_{\min}$, and $\langle N \rangle := (1 + N^2)^{1/2}$. This particular bound could be dispatched in three or four lines from some simpler inequalities; but it took some time to come up with those inequalities, and I had to do a dozen further inequalities of this type. This is a task that seems extremely ripe for automation, particularly with modern technology.

Recently, I have been doing a lot more coding (in Python, mostly) than in the past, aided by the remarkable facility of large language models to generate initial code samples for many different tasks, or to autocomplete partially written code. For the most part, I have restricted myself to fairly simple coding tasks, such as computing and then plotting some mildly complicated mathematical functions, or doing some rudimentary data analysis on some dataset. But I decided to give myself the more challenging task of coding a verifier that could handle inequalities of the above form. After about four hours of coding, with frequent assistance from an LLM, I was able to produce a proof of concept tool for this, which can be found at this Github repository. For instance, to verify (1), the relevant Python code is

```
    a = Variable("a")
    b = Variable("b")
    c = Variable("c")
    assumptions = Assumptions()
    assumptions.can_bound((a * b * c) ** (1 / 3), max(a, b, c))
```

and the (somewhat verbose) output verifying the inequality is

```
Checking if we can bound (((a * b) * c) ** 0.3333333333333333) by max(a, b, c)
from the given axioms.
We will split into the following cases:
[[b <~ a, c <~ a], [a <~ b, c <~ b], [a <~ c, b <~ c]]
Trying case: ([b <~ a, c <~ a],)
Simplify to proving (((a ** 0.6666666666666667) * (b ** -0.3333333333333333)) *
(c ** -0.3333333333333333)) >= 1.
Bound was proven true by multiplying the following hypotheses :
b <~ a raised to power 0.33333333
c <~ a raised to power 0.33333333
Trying case: ([a <~ b, c <~ b],)
Simplify to proving (((b ** 0.6666666666666667) * (a ** -0.3333333333333333)) *
(c ** -0.3333333333333333)) >= 1.
Bound was proven true by multiplying the following hypotheses :
a <~ b raised to power 0.33333333
c <~ b raised to power 0.33333333
Trying case: ([a <~ c, b <~ c],)
Simplify to proving (((c ** 0.6666666666666667) * (a ** -0.3333333
333333333)) * (b ** -0.3333333333333333)) >= 1.
Bound was proven true by multiplying the following hypotheses :
a <~ c raised to power 0.33333333
b <~ c raised to power 0.33333333
Bound was proven true in all cases!
```

This is of course an extremely inelegant proof, but elegance is not the point here; rather, that it is automated. (See also this recent article of Heather Macbeth for how proof writing styles change in the presence of automated tools, such as formal proof assistants.)

The code is close to also being able to handle more complicated estimates such as (3); right now I have not written code to properly handle hypotheses such as $N_{\max} \sim N_{\mathrm{med}} \sim N$ that involve complex expressions such as $N_{\max} = \max(N_1, N_2, N_3)$, as opposed to hypotheses that only involve atomic variables such as N_1, N_2, N_3, but I can at least handle such complex expressions in the left and right-hand sides of the estimate I am trying to verify.

In any event, the code, being a mixture of LLM-generated code and my own rudimentary Python skills, is hardly an exemplar of efficient or elegant coding, and I am sure that there are many expert programmers who could do a much better job. But I think this is proof of concept that a more sophisticated tool of this form could be quite readily created to do more advanced tasks. One such example task was the one I gave in the above MathOverflow question, namely being able to automatically verify a claim such as

$$\sum_{d=0}^{\infty} \frac{2d+1}{2h^2(1+\frac{d(d+1)}{h^2})(1+\frac{d(d+1)}{h^2m^2})^2} \lesssim 1 + \log(m^2)$$

for all $h, m > 0$. Another task would be to automatically verify the ability to estimate some multilinear expression of various functions, in terms of norms of such functions in standard spaces such as Sobolev spaces; this is a task that is particularly prevalent in PDE and harmonic analysis (and can frankly get somewhat tedious to do by hand). As speculated in that MO post, one could eventually hope to also utilize AI to assist in the verification process, for instance by suggesting possible splittings of the various sums or integrals involved, but that would be a long-term objective.

This sort of software development would likely best be performed as a collaborative project, involving both mathematicians and expert programmers. I would be interested to receive advice on how best to proceed with such a project (for instance, would it make sense to incorporate such a tool into an existing platform such as SageMATH), and what features for a general estimate verifier would be most desirable for mathematicians. One thing on my wishlist is the ability to give a tool an expression to estimate (such as a multilinear integral of some unknown functions), as well as a fixed set of tools to bound that integral (e.g., splitting the integral into pieces, integrating by parts, using the Hölder and Sobolev inequalities, etc.), and have the computer do its best to optimize the bound it can produce with those tools (complete with some independently verifiable proof certificate for its output). One could also imagine such tools having the option to output their proof certificates in a formal proof assistant language such as Lean. But perhaps there are other useful features that readers may wish to propose.

Comments (30)



danielstone010 · 1 May 2025

Estimators and ancillary equations… I really do love this stuff.



hideoutleftca8790a9b5 · 1 May 2025

Subject: Suggestions for Advancing Your Asymptotic Estimate Verifier

Dear Professor Tao,

Thank you for sharing your insightful post and proof-of-concept tool for verifying asymptotic estimates. Your vision for automating this tedious yet critical task in mathematical analysis is inspiring, and your prototype demonstrates exciting potential. As someone deeply interested in computational mathematics, I'd like to offer some constructive suggestions for developing this project further, addressing your questions about collaboration, platform integration, and desirable features. Collaboration Structure To realize a robust verifier, a collaborative team combining mathematicians, programmers, and computational experts would be ideal. I suggest: • Team Composition: Include analysis experts (e.g., in PDEs or harmonic analysis) to define inequality types and test cases, software engineers proficient in Python and optimization (e.g., linear programming with CVXPY), and specialists in computer algebra systems (e.g., SageMath) or formal verification (e.g., Lean). AI experts could enhance heuristic suggestion features. • Workflow: Adopt an agile development model with sprints focused on specific features (e.g., handling sums, functional estimates). Regular virtual workshops could align mathematical requirements with technical constraints. • Community Engagement: Host the project on GitHub to encourage open-source contributions. Outreach via MathOverflow, X, or conferences like AMS meetings could attract contributors. A hackathon to tackle specific challenges (e.g., sum verification) could accelerate progress. Platform Integration Integrating with SageMath seems a natural fit, given its open-source nature, Python-based ecosystem, and support for symbolic computation via SymPy and numerical tools via NumPy. SageMath's active mathematical community would facilitate adoption and contributions. Key steps include: • Refactoring your prototype to leverage SageMath's symbolic manipulation for complex expressions (e.g., N_{\max} = \max(N_1, N_2, N_3)). • Adding a SageMath package for your verifier, with tutorials to ease onboarding for mathematicians. • Optionally, integrate a Lean backend to output formal proof certificates, leveraging mathlib's real analysis capabilities. This could appeal to users seeking rigorous verification. While Mathematica offers powerful symbolic tools, its proprietary nature may limit collaboration. SymPy alone could suffice for a lightweight standalone tool but lacks SageMath's comprehensive ecosystem. Desirable Features Based on your wishlist and the needs of the analysis community, I propose the following features for a general estimate verifier: 1. Intuitive Input: Support LaTeX-like input for inequalities, sums, and integrals via a GUI or Jupyter notebook interface. Allow users to define constraints (e.g., N_{\max} \sim N) and function spaces (e.g., L^p). 2. Tool Specification: Enable users to specify allowed bounding techniques (e.g., Hölder's inequality, AM-GM) from a library of standard inequalities, extensible with custom rules. 3. Optimization: Automatically minimize constants in \lesssim bounds using linear programming or numerical optimization. AI-driven suggestions for case splits or bounding strategies could enhance efficiency. 4. Proof Output: Generate human-readable LaTeX proofs and machine-checkable certificates in Lean or Coq. A high-level proof summary alongside detailed steps would balance readability and rigor. 5. Complex Expressions: Handle sums (e.g., your MathOverflow example) via asymptotic approximations or numerical bounds, and support functional estimates with a database of inequalities (e.g., Sobolev's). 6. Counterexamples: For false inequalities, provide asymptotic regimes where the bound fails, validated numerically or symbolically. 7. Visualization: Offer plots of asymptotic behaviors or proof trees to aid intuition. Development Roadmap A phased approach could structure development: • Phase

1 (6-12 months): Enhance your prototype to handle complex constraints (e.g., N_{\max} \sim N), add a basic GUI, and integrate with SageMath. Implement a library of common inequalities. • Phase 2 (12-24 months): Support sums and functional estimates using asymptotic methods and AI-suggested strategies. Add numerical validation for robustness. • Phase 3 (24+ months): Implement Lean proof output and optimize bounds. Release as a polished SageMath package with extensive documentation. Specific Suggestions for Your Examples • Arithmetic Inequality (abc)^{1/3} \lesssim a + b + c: Your case-splitting approach is robust. Adding AM-GM as an alternative proof path could yield tighter constants and demonstrate the tool's flexibility. • Complex Estimate (Equation 3): Extend parsing to handle N_{\max} and \sim by defining approximate equalities as bounded ratios (e.g., c_1 N \leq N_{\max} \leq c_2 N). Numerical tests in representative cases could guide case splitting. • Sum Estimate: For the infinite sum, automate integral approximations (e.g., via Laplace's method) and regime splitting (e.g., small h, large m). SageMath's sum function could validate bounds numerically. Final Thoughts Your prototype is a compelling proof of concept, and with collaborative development, it could become a transformative tool for mathematicians. SageMath integration, a user-friendly interface, and formal proof output would maximize impact. I'd be thrilled to contribute ideas, test cases, or even code snippets to explore specific features (e.g., sum verification). Please feel free to share updates or specific challenges—I'd love to stay engaged with this exciting project! Best regards,

[Going forward, I am requiring that AI-generated text be explicitly labeled as such in order to be approved as a comment. -T.]

 **Lars Ericson** · 1 May 2025

You may be interested in the work of the Exact Computation Group of Chee Yap which uses tools like Sturm Sequences, Grobner Bases and numbers represented as solutions of polynomial systems to give exact answers to inequalities on expressions involving real algebraic numbers. See

Click to access isItZero.pdf

leading to

https://cs.nyu.edu/~exact/papers/

Get Outlook for Androidhttps://aka.ms/AAb9ysg

 **Anonymous** · 2 May 2025

Isn't proving general inequalities (even a single one inequality) related to the theory of reals? Wouldn't the complexity issues there affect?

 **Terence Tao** · 2 May 2025

Note here that I am not interested in exact (infinite precision) inequalities, but rather in *estimates* – inequalities "up to constants". This is a somewhat different language from the language of the reals, and in fact closer to tropical arithmetic. (Indeed, for this toy problem, one could conceivably actually phrase everything inside a tropical ring, and appeal to an existing tropical algebra package, but I am looking to generalize to other contexts – e.g., expressions involving more transcendental functions such as

exponentiation and logarithm – in which the language of tropical algebra might be insufficient.) This is simpler in many ways, most notably due to the equivalence up to constants $X + Y \sim \max(X, Y)$ of addition and maximum, which allows one to greatly simplify many expressions into monomial form (once one knows how things are ordered).

Related to this (and your other comment), whereas an exact inequality can be refuted with a single example, an asymptotic estimate requires a parameterized family of counterexamples. One possible desirable feature – not currently implemented – is to have the tool output a specific such asymptotic counterexample whenever it is unable to verify the estimate. (This is presumably possible using dual linear programming to help locate the certificates, but I haven't seriously attempted to do this yet.)

 **Anonymous** · 2 May 2025

I mean existence of a violation means invalid inequality. Since you care about proving truth about only one inequality at a time, wouldn't Collins procedure typically run in time cubic or quartic in number of bits of required precision (assuming number of variables is O(1))?

 **Anonymous** · 2 May 2025

In the inequalities and probability discussions that Alexander Bogomolny posted on his blog https://x.com/CutTheKnotMath it was often clear that he viewed some of the problems as art of proof and art of intuition. Interestingly enough it now comes to a focus in foremost mathematical questions on AI abilities to discover mathematical results and evolving more in an art form

 **Lawrence Paulson** · 2 May 2025

For the special case of real asymptotics (including Landau symbols) an automated package is already available, thanks to the awesome Manuel Eberl. Its main mode of usage is to confirm (within Isabelle's proof kernel) a limit obtained elsewhere, e.g. through a computer algebra system, although it can also calculate limits itself. It can also prove that various properties hold in the limit, with a rather general notion of "in the limit" obtained using filters.

 **Anonymous** · 2 May 2025

would a formalization of the ideas sketched in https://mathoverflow.net/questions/154373/differential-galois-number-theory help?

 **Anonymous** · 2 May 2025

You may be interedted in https://chatgpt.com/canvas/shared/6814dcdedae481919d4c964263d602a2

 **-** · 2 May 2025

Hi Terry, thanks for the wonderful post. I was the OP on that Math overflow question. My students have already built this tool to solve these types of questions. The first is through API calls to a math-specialized LLM, and the second by a modification of Alpha Geometry to questions like these. In particular, we are able to obtain full proofs to estimates like these in both questions. We will be emailing you shortly with more details. Thanks so much for the wonderful idea, we've had a wonderful year building these tools.

Terence Tao · 2 May 2025

That's great! I look forward to seeing the final products your group has produced.

Guanyuming He · 2 May 2025

Dear Prof Tao,

A few noob questions:

1. Does $a \lesssim b$ mean something like the Big-O in complexity theory, i.e., there exists $C > 0$ such that $a \leq Cb$ (except here we don't use absolute value, as everything is positive)? I asked because, as a non-native speaker, your sentence "*we are willing to lose an unspecified constant in the estimates*" seems to me meaning adding a constant $a \leq b + C$ instead.
2. To attack your example inequality by cases, we rely on the observation that $a + b + c \sim \max(a, b, c)$. What about equality involving other operations, mult, exp, division, etc.; do they rely on such observations, too? And where could one find a list listing such an observation for each operation?

Thank you,
Guanyuming He

Terence Tao · 2 May 2025

1: Yes; in this context, "constants" refer to "multiplicative constants" rather than "additive constants".

2: In general multiplicative operations (including raising to a fixed power) are easy to handle, since after taking logarithms, they become linear expressions, and in particular the question of determining whether one inequality implies another can be handled by linear programming tools.

Terence Tao · 2 May 2025

I am belatedly realizing that to make problems like this easy for brute-force computer tools to solve (as opposed to human verifiers), it makes sense to break up compound statements involving few variables into many atomic statements involving many variables, by giving each relevant term its own name. For instance, if one wants to prove that $\max(a,b)^* \min(c,d) \lesssim e$, the helpful thing to do from a computer's point of view is to introduce new variables $f = \max(a,b)$, $g = \min(c,d)$, $h = \max(a,b)^* \max(c,d)$, and reformulate the problem as that of establishing the inequality $h \lesssim e$ subject to the relations $f \sim \max(a,b)$, $g \sim \min(c,d)$, $h \sim f^* g$. The point is that case splitting becomes significantly more straightforward: for instance, the relation $f \sim \max(a,b)$ can be decomposed into the relations $f \gtrsim a$ and $f \gtrsim b$, together with one of $f \sim a$ and $f \sim b$ (and one has the option to split into cases to obtain one of these). My current implementation simply splits all cases at once, but one can do a more targeted approach of using linear programming to see whether the non-split constraints are enough to establish the desired bound, and if not perform one case split and recurse. (Another efficiency gain occurs if the same term appears multiple times in an expression, as it can be assigned just a single variable.)

 **Terence Tao** · 2 May 2025

Another thing I am realizing is that while humans prefer to see the steps of a logical argument arranged in a linear sequence, with the justification of line n preferably located in a nearby preceding line (such as line $n-1$ or line $n-2$), computers have no such restrictions, and can easily manage large collections of hypotheses as an unordered set, with justifications drawn from arbitrary portions of such set as needed without any requirement to arrange things in any sort of linear order. In particular, the paradigm of proving estimates such as $X \lesssim Y$ via chains of sub-estimates $X \lesssim Z_1 \lesssim Z_2 \lesssim \dots \lesssim Z_n \lesssim Y$, while helpful for human readers, may not be the right paradigm for proofs that are to be fully automated; I am leaning more towards just dumping a large number of relevant sub-estimates into an unordered set and calling a linear program at the end to obtain the desired conclusion from this unordered set of hypotheses.

>  **Andrew Krause** · 3 May 2025
>
> I don't know what the right generalization of the earlier idea of phrasing everything in a tropical ring would be, but there may be something clever to be done in using such a formalism. Namely, given some way of representing the set of inequalities you are conceivably interested in, one could think about building a graph (possibly a tree) of inequalities which would then be itself a valuable database that could be queried quickly and expanded as needed (rather than needing to "re-prove" a given atomic inequality many times).
>
> This may be a naive idea, but something like this is used in some software packages that perform Symbolic Regression.

 **Evan Conway** · 2 May 2025

For proving such asymptotic inequalities (with constant-loss), rule-based integration systems like Rubi are a promising blueprint to follow. Ideally, one could specify similar rules for inequalities, along with a list of known inequalities to try to reduce to.

My intuition is that automatically finding a counterexample would be more difficult, but I am unsure of that. It may be tractable to use the same kind of rule-based system to automatically find a counterexample.

>  **Terence Tao** · 2 May 2025
>
> Thanks for the pointer! Yes, I would like eventually to move to a rules-based framework, where one has the option to either (a) manually select rules to transform the proof state, (b) invoke a brute-force routine to try to automatically search the entire space of possible "moves", or (c) use some sort of AI to suggest a given rule to apply for any given state. A complex suite of estimates to be proven could then eventually be handled by some combination of (a), (b), and (c), for instance by brute forcing a few simple cases to discern some initial pattern, then (based on those preliminary findings) doing some medium difficulty cases by hand, calling an AI to then handle as many of the other cases as it can, and then finally study the last remaining cases manually.
>
> EDIT: actually a "tactic"-based framework may be a more accurate description of my long-term vision than a rules-based framework, using modern proof assistant languages

such as Coq or Lean as models. (One could in fact try to encode such a verifier within say Lean metaprogramming, for instance leveraging existing tactics such as `linarith`... hmm, that actually sounds like a good way to go, now that I think about it.)



Dave Doty · 3 May 2025

This looks like a great idea! Someone above mentioned the sympy package; I'm not sure if you have experience with it.

Looking through your repo, it seems plausible that you are manually reproducing some of what sympy can already handle. Not the asymptotics analysis to my knowledge... I just mean for instance your `Variable` and `Expression` classes seem very similar to `sympy.Symbol` (https://github.com/sympy/sympy/blob/07449a736d039ce9bfca9bd55fdfc073fe4c05fb/sympy/core/symbol.py#L212) and `sympy.Expr` (https://github.com/sympy/sympy/blob/07449a736d039ce9bfca9bd55fdfc073fe4c05fb/sympy/core/expr.py#L47), and subclasses thereof, e.g., `sympy.Mul` similar to your `Mul` class (https://github.com/sympy/sympy/blob/07449a736d039ce9bfca9bd55fdfc073fe4c05fb/sympy/core/mul.py#L91).

Long-term, integrating such a tool directly into sympy would be great. In the meantime, it seems likely that your tool would be easier (both for you to develop and for users to use) using sympy objects, in order to leverage all the power sympy already has. For instance it can tell if an expression is a polynomial and then convert it to a polynomial object to do polynomial-related things, or render nice LaTeX-quality math expressions in a Juypyter notebook. For an example of the latter, see here: https://github.com/UC-Davis-molecular-computing/ode2tn/blob/main/notebook.ipynb, where the differential equations rendered below the 2nd cell are from executing this function: https://github.com/UC-Davis-molecular-computing/gpac/blob/bf2f0ae1bcc4254e8b88909a0f481b11e126cd1b/gpac/ode.py#L876

I also left an issue on the GitHub issues page describing one source of potential bugs: https://github.com/teorth/estimates/issues/1



Terence Tao · 3 May 2025

Thanks for the suggestion! I was not familiar with sympy but it certainly sounds relevant. My particular symbolic variables basically take values in a tropical algebra rather than a traditional algebra (for instance, I have no notion of subtraction), but it looks like the general formalism sympy uses is certainly very compatible.

In any case, I completed a more sophisticated version of the code, which is now a bare-bones proof assistant, able to prove various claims in both propositional logic and asymptotic estimates through the application of various tactics (such as "simplify all expressions", "split into cases", "prove by contradiction", "apply linear programming", etc.). I even have an "autosolve()" routine which should in principle solve any implication of the type I had in mind, although the number of cases grows exponentially in the number of "max" or "min" (or addition) operations that are in play, so I would imagine in practice one would want a more human-guided (or possibly AI-guided?) deployment of tactics to keep proofs at civilized lengths. Examples of the code in action can be found at the Github repository.

I'll have to turn attention to other things for a while, so I won't be quite so obsessively developing this code at the rate of the last two days. I'll try at some point to understand

how sympy works more. Probably the first task would be to port over the symbolic linear programming tool I wrote (basically a wrapper around Z3's exact linear programming solver) to sympy, if it doesn't already have native linear programming capability.

> **Dave Doty** · 4 May 2025
>
> I'd be happy to contribute to this. If you did want help, it would help me at some point (no rush) to verbally discuss your existing implementation so I can ensure I understand the intent of various parts and faithfully translate. I could also help with the stuff that would make this package easy to install and use, e.g., so users can do `pip install asymptotic-estimates` (or whatever you want to call the package; "estimates" is taken though: https://pypi.org/project/estimates/)
>
> That said, I don't really know anything about computer algebra. I'm sure some authors of sympy are much more experts about the relevant techniques and algorithms involved. (I do understand asymptotic analysis fairly well though.) I mainly am familiar with sympy because I use it for some packages I've written to help with things like integrating and plotting ODEs and simulating chemical reaction networks (and plotting their trajectories).
>
> Regarding the linear programming question:
>
> It seems sympy can do (numeric) linear programming (I ran your linear programming examples and it seems they are numeric, i.e., you do not appear to be trying to do some sort of "symbolic" solution to a linear program): https://docs.sympy.org/latest/modules/solvers/solvers.html#sympy.solvers.simplex.lpmin
>
> I could not see a way to directly get a certificate of infeasibility from that or related functions in sympy. One could certainly still use the Z3 solver, converting sympy `Symbol`'s to Z3 variables as you do a conversion from your custom `Variable` class, or for more general expressions, something like this: https://stackoverflow.com/questions/22488553/how-to-use-z3py-and-sympy-together
>
> I don't have experience with Z3. It's (to my understanding) a sort of mish-mash of many practical theorem-proving and optimization tools, some SAT-solving, some linear programming, probably other fancy things. If you just want a Python tool for linear programming, I'd guess scipy is the simplest (but I don't know): https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html#scipy.optimize.linprog
>
> I used Google's or-tools (https://developers.google.com/optimization) many years ago, which is a front-end for multiple back-end optimization solvers, e.g., for integer programming, the commerical Gurobi product (https://www.gurobi.com/) or the open-source SCIP solver (https://www.scipopt.org/). But this is more complex and usually requires separate installation of the back-end solver (as opposed to using scipy, which is a self-contained Python package that can be declared as a dependency of this package and automatically get installed when this is installed).
>
> (To be honest, I'm not sure how the linear programming stuff is related to the asymptotic analysis that seems to be the main goal of the package.)

I'm primarily recommending sympy to replace your classes such as `Variable`, `Expression`, `Mul`, etc., since what appear to be similar classes exist in sympy. But I haven't yet wrapped my head around exactly what those are doing. For example, the `simp` method of many of these seems like it is doing a particular kind of asymptotic-analysis-relevant simplification, for example removing constants (e.g., changing 3*x to x if I understand), which is not something I imagine sympy implements. So this would become a function in your package (e.g., `asymptotic_simplify`) that operates on sympy `Expr` objects.



Terence Tao · 4 May 2025

Thanks for this! I did experiment with a number of other LP packages such as Pulp, but Z3 was the only one I could find which could do exact linear programming in rational arithmetic (no floating point rounding issues), which in particular allowed one to reliably work with strict inequalities as well as non-strict ones in the constraints. In principle it would be nice to also be able to do exact linear programming in other totally ordered fields with computable arithmetic (e.g., algebraic numbers such as $\sqrt{2}$), but rational is fine for now (it means that I can only raise my asymptotic expressions to rational powers, but this already covers a lot of use cases). It shows up in asymptotic analysis because many asymptotic estimates can be deduced as (log)-linear combinations of other asymptotic estimates. For instance, $X^2Y^3 \lesssim 1$ and $X^5Y^2 \lesssim 1$ implies $X^9Y^8 \lesssim 1$ because $(9,8) = 2 \times (2,3) + (5,2)$ is a non-negative linear combination of $(2,3)$ and $(5,2)$.

The linear programming code is essentially decoupled from the rest of the code though – it is called by a single log_linarith() routine. In order for it to benefit from sympy integration, one would like to have a symbolic linear programming tool that works in any computable ordered field, which is an interesting project but somewhat orthogonal to my current goal.

On looking at sympy's code it does indeed seem that many of my classes are basically imitating classes in sympy: my Type imitates sympy's Basic, my Statement roughly corresponds to (I think) sympy's Boolean, my defeq() routine seems to correspond to sympy's equality relation ==, and so forth. My Variable and Expression classes are special cases of sympy's Symbol and Expr classes where they are supposed to take values not exactly in the positive reals, but in some sort of order of infinity semiring (as I just discussed in my most recent blog post) so they have some of the usual algebraic properties such as commutativity and associativity of multiplication as well as more exotic laws such as the law $3 = 1$ you mentioned, but on the other hand lack a notion of subtraction or a zero. I did indeed put extra functionality such as simp() into the objects because it was a convenient location to place each component of the simplifier, but I could indeed just have a giant external asymptotic_simplify() routine instead that recursively traverses through the symbol tree and does various things depending on the type of symbol matches to.

The symbolic expressions and statements are themselves the building blocks of goals (which are a set of hypotheses, together with a desired conclusion), which are in turn the components of a proof state (which is a set of goals). But the basic tooling (or "tactics") for these latter objects should largely be able to treat the symbolic objects as black boxes – they basically just call some key methods of these objects, such as the simplifier, in specific ways. On e technical issue I found though was that in order to be able to form sets of expressions, I could not modify the == operation (it stopped hashing from working correctly, or something). Which is why I ended up having a separate method defeq() for symbolic equality. This leads to the situation where two instances of the same expression, e.g., $x + y$ and $x + y$, might be distinct objects (stored in distinct locations in memory) despite being symbolically (or definitionally) equal. I don't quite understand how sympy deals with this problem – can one work with sets of sympy symbolic expressions that are automatically reduced with respect to symbolic equality (so, for instance, a set would never contain two separate copies of $x + y$)? This caused some issues in my code because tactics like "gather all the terms appearing in the hypotheses and conclusions into a set, and perform linear programming using variables associated to these terms" become a little subtle to implement if symbolically identical terms become distinct for the purposes of forming a set (I ended up making some custom replacements for the base Python set operations to try to enforce symbolic distinctness of elements). (Maybe if I understood what "hashable" really meant, I could fix this problem more elegantly, but in any case I got the code to work.)



Aaron Meurer · 4 May 2025

To answer the question in your final paragraph, in SymPy a set containing two structurally equal expressions would only contain the expression once. The way this is achieved is firstly, all expression objects are immutable (this is important because otherwise they cannot be hashed). If you want to "change" an expression you instead create a new one. In Python, if an object is immutable, you can define a hash for it, which is just an int that is somehow determined by whatever is defined in the expression (this is done by defining the __hash__ method). The number can be anything, so long as if two objects are equal, their hashes are also equal (a == b should imply hash(a) == hash(b)). A typical way to do this for a symbolic tree is to define the hash as something like hash(args) where args is a tuple of the subexpressions, and more generally you typically would just hash whatever you use to create the object (the arguments to __init__()).

Finally, the == (__eq__) method should return True or False. The Python set and dict objects use hash tables based on this hash object. They fallback to == when the hash values are equal in case of hash collisions, which is why == has to be used as a boolean True/False structural equality check. In SymPy we have a separate Eq() object to represent a symbolic mathematical equality. It is technically possible to use == as syntactic sugar in Python, but we avoid it for this reason.

Glancing at your code, I think your main issue is that you aren't defining __eq__, so it just uses the Python default, which is to only consider two objects to be equal if they are exactly the same object. But if define __eq__ (and __hash__ accordingly), then Python will treat two different objects that both represent the exact same thing as if they were the same object.



Dave Doty · 4 May 2025

> In principle it would be nice to also be able to do exact linear programming in other totally ordered fields with computable arithmetic (e.g., algebraic numbers such as sqrt{2}), but rational is fine for now

I tried and sympy won't let you use symbolic transcendentals like sqrt(2) in its linear programming solvers. (Though it supports them more generally for other symbolic manipulation.)

> The symbolic expressions and statements are themselves the building blocks of goals (which are a set of hypotheses, together with a desired conclusion), which are in turn the components of a proof state (which is a set of goals). But the basic tooling (or "tactics") for these latter objects should largely be able to treat the symbolic objects as black boxes – they basically just call some key methods of these objects, such as the simplifier, in specific ways. On e technical issue I found though was that in order to be able to form sets of expressions, I could not modify the == operation (it stopped hashing from working correctly, or something). Which is why I ended up having a separate method defeq() for symbolic equality. This leads to the situation where two instances of the same expression, e.g., x+y and x+y, might be distinct objects (stored in distinct locations in memory) despite being symbolically (or definitionally) equal. I don't quite understand how sympy deals with this problem – can one work with sets of sympy symbolic expressions that are automatically reduced with respect to symbolic equality (so, for instance, a set would never contain two separate copies of x+y)?

I'm not sure actually. I'm trying to figure out if you want this for just speed, or also for correctness.

In fact this code I wrote was somewhat confusing to me:`print(2*x + 2*y == 2*(y + x)) print((2*x + 2*y) is 2*(y + x))`

== is the Python operator that calls a method called `__eq__` on the two objects to see if they are equal by value (i.e., represent the same data), whereas the `is` keyword is supposed to represent so-called "referential equality", i.e., the expressions refer to the same object in memory. I would have thought it would print True (since those two expressions represent the same abstract mathematical quantity) then False (since they are represented by distinct objects in memory), but it actually prints True for both.

> This caused some issues in my code because tactics like "gather all the terms appearing in the hypotheses and conclusions into a set, and perform linear programming using variables associated to these terms" become a little subtle to implement if symbolically identical terms become distinct for the purposes of forming a set (I ended up making some custom replacements for the base Python set operations to try to enforce symbolic distinctness of elements). (Maybe if I understood what "hashable" really meant, I could fix this problem more elegantly, but in any case I got the code to work.)

Hashable means, literally, that the object implements a method called `__hash__` taking no arguments (other than self as all methods take) returning an int.

TLDR: if you object is immutable (cannot be changed after being created), then you can fairly easily make it hashable, and otherwise it probably should not be hashable.

What it's supposed to mean is that not only is this method implemented, but it guarantees that any two objects that are "semantically equal", i.e., considered equal by the `__eq__` method (which is what is called when you write `a == b`, shorthand for `a.__eq__(b)`), output the same integer (called a hash value). In particular is should be the case that the hash value of an object never changes during its lifetime. (even if some of its fields change, so long as those fields are not used in computing the hash value)

The idea is that some collections such as `set` and `dict` use the hash value to determine when two objects are semantically equal, i.e., represent the same data whether or not they are literally the same object in memory. For instance I could allocate two strings in memory, at different locations, but both are three letters: "abc". They are semantially equal but not referentially equal.

More precisely, it uses the hash value to conclude decisively when two objects are semantically unequal: if the contract above is obeyed, then if their hash values are unequal, they are guaranteed to represent different data, so one can skip a (potentially expensive) check on all their data to see if all those data are equal. If the hash values are equal, the objects are not guaranteed to be semantically equal, but if the `__hash__` method is written well then this will happen infrequently.

To be concrete, actually sympy violates this contract (I'm a bit surprised actually, I'd have expected them to be more careful.)

sympy implements a class `Symbol` with a field `name`. If `Symbol` were immutable (which it is not in sympy, to my surprise), I need to specify the name of the Symbol when I create it and would not later be able to change it. If `Symbol` is mutable (as it is in sympy), then I can do something like this:

```
x = Symbol("x")
print(x)
```

```
x.name = "y"
print(y)
```

which prints “x”, then “y”. The problem is this:

```
from sympy import Symbol
x = Symbol("x") symbol_set = {x} # create singleton set with only x
print(f"is symbol {x} in the set? {x in symbol_set}")
x.name = "y"
y = Symbol("y")
print(f"is symbol {x} in the set? {x in symbol_set}")
print(f"is symbol {y} in the set? {y in symbol_set}")
```

Consider what happened above. We create a symbol variable x, representing the symbol whose name is “x”. We add it to the set then ask is it there. Of course it is. So the first print statement is “is symbol x in the set? True”.

The problem is the next few lines. We change the name of the symbol x to “y” even though in Python the variable is still called x. But despite its variable name in the Python source code, it now represents a mathematical variable named “y”. We then ask is that symbol in the set? The abstract correct answer should be no: we only added a symbol named “x”.

Yet it prints “is symbol y in the set? True”. It is true that the object we inserted into the set is in that set. However, that object no longer represents a symbol named “x”; it now represents a symbol named “y”.

The final print statement confirms that if we create a dedicated symbol that represents the name “y” (and has only ever represented that value), that object is not in the set: “is symbol y in the set? False”

This is all a bit subtle, but the basic idea is that we want membership in collections such as sets (or keys in dictionaries) to be based on semantic equality (i.e, do these two objects, which may not be the same object in physical memory, represent the same data) rather than referential equality (are these literally the same object occupying the same address in memory?)

Being “hashable” means more or less that your data type respects this.

How I do this in practice tends to be: if my object is immutable (cannot be changed after being created) I let it be hashable, otherwise not.

Anyway we are a bit into the weeds here, perhaps a Zoom call at some point in the next few weeks we could clear some of this up.



Aaron Meurer · 4 May 2025

Hello Dr. Tao.

I am one of the SymPy developers. I noticed your post about this project (quite by coincidence) today and that you mentioned you might be interested in integrating this with SymPy.

Being able to better determine if inequalities are true or not is something that we've very much been been wanting to improve in SymPy. We already have some rudimentary support for integration with Z3 https://github.com/sympy/sympy/blob/2461d96714033b1f2054aca7283a87a8755693ce/sympy/logic/algorithms/z3_wrapper.py, as well some support for making deductions about linear inequalities https://github.com/sympy/sympy/blob/2461d96714033b1f2054aca7283a87a8755693ce/sympy/logic/algorithms/lra_theory.py, both of which were implemented by a Google Summer of Code student in 2023. However, support for anything more than that is very limited.

It seems like your tool is primarily designed to prove asymptotic inequalities. One thing I'm trying to figure out is whether that's something that SymPy would find useful. My guess is it would, but I'd need to think about it. Some of the examples in your project README are just direct inequalities (the multiplicative constant is 1). Does your routine give an estimate of the constant for <~ inequalities?

I've informed the SymPy community about this on the SymPy mailing list https://groups.google.com/g/sympy/c/ndN7CEyPgCY



Terence Tao · 4 May 2025

Good question! The proof system I have is not well optimized to track constants "in real time", especially when one wants to do things like argue by contradiction and replace a conclusion such as $X \lesssim Y$ with the negated hypothesis $X \gg Y$. However, once one has used this asymptotic proof system to obtain a desired goal (e.g., to establish one asymptotic estimate as a consequence of several asymptotic hypotheses), one could in principle then perform some "proof mining" and inspect the atomic steps of the proof to see how the constants in the various intermediate asymptotic estimates appearing in the argument had to be related to each other (e.g., if we used $X \lesssim Y$ and $Y \lesssim Z$ to deduce $X \lesssim Z$, then the implied constants $C_{X,Y}$, $C_{Y,Z}$, latex $C_{X,Z}$ attached to these estimates obey the relation $C_{X,Z} \leq C_{X,Y} C_{Y,Z}$). By chaining together all these relations (and applying a linear program, in more complicated cases) one can then figure out what the optimum final constant is as a function of the initial constants. This might be a nice byproduct of these sorts of tools – currently in analysis papers, one usually doesn't bother to work out the explicit constants because so much tedious bookkeeping is involved. [This would require upgrading the proof state object to not only track the current proof state, but the entire history of steps taken to get to that state from the initial state, but that looks doable with a reasonable amount of effort.]

After looking a bit more at sympy it seems like it is moderately beneficial for me to refactor the code to use the base sympy structures, particularly while the codebase is still relatively small, but that because my symbolic expressions are in a rather exotic type of space, with slightly nonstandard rules of algebra (and no subtraction), I wouldn't be able to use the more advanced features of sympy. Still, I had some ad hoc code which could certainly be replaced by existing core sympy methods (and in particular it seems the way sympy handles equality

is significantly cleaner than the way I hacked it out). I'll probably make that refactor my main goal over the next week or so.

Eventually I plan to work with more complicated data types than the current "order of magnitude" – for instance, to deal with functions in various function spaces such as Sobolev spaces, but it looks like the basic sympy framework would be able to handle those as well. Hopefully this package can take care of various "housekeeping" tasks on the expressions involved, such as simplifying them into a normal form, so I can focus more on the tactics manipulating the proof state, which is my main interest.

 **nunosempere2** · 3 May 2025

Yeah, as a programmer I a) find it very impressive that you can do what the OP is doing from scratch, even with LLM assistance, but b) suspect it would be more meaningful/parsimonious to incorporate that functionality into an already existing project.

 **Orders of infinity | What's new** · 4 May 2025

[...] ability to prove asymptotic implications by (log-)linear programming; this was implicitly used in my previous post. One can also use the language of (log-)linear algebra to describe further properties of various [...]

 **Anonymous** · 8 May 2025

Another tool you might want to look at for inspiration is Gappa:

https://gappa.gitlabpages.inria.fr/